

Agent-Aware Networking

In-Network Classification and Differentiated Transport QoS for AI-Agent Traffic on a Programmable Data Plane

BCSE308L/P Computer Networks · Research Project — Detailed Project Description · August 2026

Goal: working prototype + Mininet/BMv2 evaluation + IEEE-format research paper for publication

1 · Project Overview

One-liner. A network switch (P4/BMv2 or eBPF pipeline) that recognizes AI-agent traffic — MCP tool-calls, A2A flows, agentic browsers — per-packet at line rate and, in the same forwarding pipeline, assigns it a dedicated transport treatment: its own queue, scheduler share, DSCP/ECN marking, and admission control. Bursty, machine-paced agent flows get a controlled lane instead of degrading human interactive traffic.

Outcome. A research paper (IEEE format) presenting the system, a working prototype, and a four-baseline experimental evaluation in Mininet/BMv2 — targeting publication. The paper's headline result: measurable tail-latency protection for human traffic under agent-traffic bursts (target $\geq 30\%$ p99 reduction) at negligible data-plane cost ($< 5\%$ added per-packet latency).

Why now. AI-agent traffic is a 2025–2026 phenomenon with verified data behind it, the research community has only reached the detection stage, and no existing work performs agent classification in the forwarding path or couples it to QoS — a clear, defensible research contribution.

2 · Motivation (verified statistics)

The internet's traffic mix has structurally changed. All figures verified against primary sources (Aug 2026); phrasing reflects the exact scope of each claim.

- **53% of internet traffic is automated** (Thales 2026 Bad Bot Report, 13th edition, full-year 2025 data): bad bots 40% (up 3 points YoY), benign automation 13%, human traffic down to 47% from 49%.
- **AI agents have emerged as a third traffic category** alongside traditional good and bad bots — interacting directly with applications and APIs, running real browsers with valid fingerprints and human-pattern interaction, defeating classic detection.
- **AI-driven bot attacks surged 12.5× in 2025 vs 2024** — daily blocked attacks rose from 2 million to 25 million. (Scope note: 12.5× refers to attack volume specifically, not all agent traffic.)
- Operators and standards bodies are actively discussing differentiated treatment of AI/agent traffic; a concrete citation will be pinned before paper submission.

The concrete harm: an agent fires 50 parallel MCP tool-calls across the same link as a human video call. Every switch on the path sees 51 indistinguishable TLS flows on port 443 and queues them together; the human's packets wait behind the machine-paced burst, and that queuing delay *is* the p99 latency spike. The network cannot fix what it cannot distinguish.

3 · Problem Statement

Existing QoS machinery (DiffServ, port/application-based classification, endpoint-set DSCP) cannot recognize AI-agent traffic: agents use real browsers, valid TLS fingerprints, and standard ports. Existing agent-detection solutions (Cloudflare signed agents / Web Bot Auth, DataDome Agent Trust, Akamai) operate only at the HTTP/CDN edge — they terminate TLS, see only traffic routed through them, react on request timescales, and offer only allow / block / rate-limit / monetize actions. **No mechanism exists inside the network forwarding path that identifies agent traffic per-packet at line**

rate and gives it differentiated transport service.

4 · Core Concepts

4.1 · Forwarding plane vs control plane vs application edge

Every switch/router splits into a **control plane** (the software brain: routing protocols, controller interaction, table updates; millisecond–second timescales) and a **forwarding plane** (data plane — the pipeline every packet physically traverses: parse headers → match tables → apply actions → queue to output port; nanosecond–microsecond timescales). The **application edge** (WAF/CDN/proxy) is a different location entirely: it terminates TLS and inspects HTTP requests after the fact. In this project the classification decision is made in the forwarding plane — per packet, at line rate, inside the device, for any traffic crossing it. P4 is to switches what code is to CPUs: we are not building a switch; we are writing the program a programmable switch runs. BMv2 is the software emulator of such a switch used for development and evaluation.

4.2 · Transport QoS as the action space

Once a flow is labeled, the action is a *service treatment*, not an access decision: DSCP/traffic-class rewrite; assignment to a per-class queue with a bounded scheduler weight; a class-specific ECN marking profile (shaping how senders' congestion control backs off); and admission control / rate shaping of new agent flows under congestion — including a **human-traffic-protection mode** that caps the aggregate agent share of a congested link. Agent traffic is never rejected; it is isolated into an engineered lane. Misclassification therefore degrades service class rather than blocking anyone — a graceful failure mode no allow/block system offers.

4.3 · The coupling is the contribution

Neither half is new alone: agent fingerprinting exists (application layer), in-network ML on P4 exists (Mousika, IIsy), and QoS queues are decades old. What does not exist anywhere is the coupling: **the switch itself recognizes “this is an AI-agent flow” and, in the same pipeline pass, changes how that packet is serviced.** The deployment property that follows is single-box incremental deployability: no internet-wide adoption, no new protocol, no third-party cooperation — one device at a choke point (campus/enterprise gateway, datacenter leaf, the switch fronting an API/LLM-serving cluster) benefits its domain immediately. One honest scope note for the paper: DSCP markings are routinely bleached at AS boundaries, so the QoS treatment is meaningful within the deploying domain — which is where the queues and the congestion live anyway.

5 · Literature Review and Research Gap

The relevant literature falls into three streams: (A) identifying agent/bot traffic, (B) running ML classification inside programmable data planes, and (C) networking research around agent protocols. Each stream supplies a building block; none performs agent classification in the forwarding path, and none couples agent identity to a QoS action. Entries below were verified against primary sources (Aug 2026).

5.1 · Stream A — Agent and bot traffic identification (application layer)

Work	Focus	Key result	Limitation w.r.t. this project
Whose Agent Are You? (arXiv 2606.20910)	Server-side multi-layer fingerprinting	97% attribution across six agent frameworks (AutoGen, Browser Use, Claude, Gemini, Operator, Skyvern) by fusing TLS/HTTP2/header/timing with browser-behavior signals	Depends on browser-level signals invisible mid-path; detection-only; instrumented-domain vantage

Work	Focus	Key result	Limitation w.r.t. this project
Nobody Knows You're an LLM Bot (arXiv 2606.30119)	Layer-wise ablation of agent attribution	Network layer alone is weak: IP-only $\approx 60\%$, JA4-only $\approx 45\%$ for multi-class agent attribution; 21 distinct JA4 values observed	Confirms the open problem this project attacks; offline analysis; no enforcement mechanism
When Handshakes Tell the Truth (arXiv 2602.09606)	JA4/TLS bot detection	CatBoost on JA4DB features: AUC 0.998, 98.6% accuracy — binary bot-vs-human from the handshake alone	Binary decision only; no agent classes; offline; no action coupling
AgentPrint (arXiv 2510.07176)	Encrypted-traffic fingerprints of LLM agents	F1 0.866 agent identification from timing/volume patterns on the user-agent link; infers user attributes (top-3 69–74%)	Framed as a passive privacy attack; single vantage; identification, not service treatment
Known By Their Actions (arXiv 2605.14786)	LLM browser-agent fingerprinting via UI traces	Model-level attribution of browser agents from interaction/UI traces	UI-layer signals; not network-visible at all
Web Bot Auth (RFC 9421; Cloudflare signed agents)	Cooperative agent identification	Cryptographic HTTP message signatures let declared agents authenticate to origins	HTTP-layer, adoption-dependent, invisible mid-path under TLS; action space is allow/block
Thales 2026 Bad Bot Report	Internet-scale measurement	53% automated traffic; AI agents as a third category; 12.5 $\times$ surge in AI-driven attacks	Motivating measurement; proposes governance, no network mechanism

5.2 · Stream B — In-network ML on programmable data planes

Work	Focus	Key result	Limitation w.r.t. this project
IIsy / “Do Switches Dream of ML?” (HotNets'19 line)	ML $\rightarrow$ match-action mapping	Pioneered mapping trained classical models (trees, SVM, NB, k-means) into switch match-action pipelines	Generic classification tasks; never agent traffic; no QoS coupling
pForest (arXiv 1909.05680)	Random forests in P4	Line-rate per-packet inference for hundreds of thousands of flows; phase-aware flow models; validated on Tofino-class hardware	Task-agnostic framework; classification is the output, not a service treatment
Mousika (INFOCOM'22)	Knowledge distillation for the data plane	Distills complex models into binary decision trees that fit switch constraints — the compilation recipe this project adopts	Same: enabling technique, different problem
Planter (arXiv 2205.08824)	Automated in-network ML	Modular one-click mapping of many model families to P4 targets; adopted as the de-facto mapping scheme by later work	Same: tooling, not an agent-traffic system
NetBeacon; Flowrest / Akem et al. 2025 (IJNM)	In-switch (encrypted) traffic classification	Tree models over packet+flow features classify encrypted traffic in real time inside programmable switches	Classes are applications/attacks, not agent identity; no transport-QoS action
ACC-Turbo (SIGCOMM'22)	In-network inference $\rightarrow$ scheduling	Online clustering of traffic aggregates coupled to programmable scheduling to deprioritize pulse-wave DDoS — the closest precedent for classification-to-action coupling in the data plane	Trigger is attack aggregates, not agent identity; objective is attack mitigation, not traffic-class QoS engineering

5.3 · Stream C — Networking research around agent protocols (adjacent)

Work	Focus	Key result	Limitation w.r.t. this project
NetMCP + SONAR (Li et al., 2025)	Network-aware MCP	Tool-invocation routing that weighs semantic similarity together with real-time QoS when agents call external tools	Application-layer tool selection: the network is context the agent reads, not an enforcer that treats agent traffic
LLM-powered agentic AI for 5G/6G (arXiv 2607.16066, survey)	Agents in network management	Surveys MCP/A2A adoption at management/control planes (O-RAN rApps/xApps, NWDAF interfaces)	Agents managing networks — the inverse of this project, which makes networks aware of agents

5.4 · Gap synthesis

Stream A detects agents but cannot act, operates at the application layer, and its best results lean on signals a switch never sees — while its own ablations show the network layer alone is unsolved. Stream B acts inside the data plane at line rate but has never targeted agent traffic; its closest action-coupling precedent (ACC-Turbo) keys on attack aggregates, not traffic identity. Stream C serves agents from the network's management plane without ever treating agent flows as a traffic class. **Gap statement (for the paper):** no prior system classifies agentic/MCP/A2A traffic in the forwarding path, and no prior system couples agent-traffic identity to transport-layer QoS treatment. This work is, to our knowledge, the first to do both in a single data-plane pipeline.

6 · Novelty — Four Distinguishing Axes

1. **Location — forwarding plane, not application edge.** Acts inside the switch, per packet, at line rate, on any traffic crossing the device; no proxying, no TLS termination, works beyond HTTP.
2. **Action — transport QoS, not access control.** Queueing, scheduling, ECN/congestion signaling, admission control; an engineered lane, not a verdict.
3. **Trigger — agent/MCP identity specifically,** not generic application classes (classic QoS literature) and not GPU-cluster AI-workload prioritization (AI-fabric work).
4. **Features — agent-protocol and behavioral fingerprints keyed directly to the QoS action in the same pipeline.** The contribution is this coupling.

Framed as paper contributions: (C1) the first characterization of agent traffic as a distinct in-network traffic class with line-rate-computable features; (C2) a data-plane classifier + QoS coupling architecture with a controller-assisted reclassification loop; (C3) a reproducible evaluation showing human-traffic tail-latency protection under agent bursts against four baselines; (C4) an adversarial analysis showing timing-based evasion is self-penalizing.

7 · Classification Parameters: Agent vs Human

Five feature families, ranked by expected discriminative weight. Families 1–3 carry the classifier; family 4 saturates against browser-driven agents; family 5 is deployment-dependent.

1. **Inter-packet timing statistics (strongest signal).** Human flows contain think time: heavy-tailed gaps of hundreds of milliseconds to seconds, high variance. Agent flows are machine-paced: near-deterministic inter-arrival times (IATs), sub-ms to tens-of-ms gaps, low coefficient of variation. Computed per-flow in P4 registers: EWMA IAT mean/variance, min/max IAT, coarse timing entropy. Key property: evasion is self-defeating — an agent inserting human-like think time becomes as slow as a human, forfeiting its own purpose.
2. **Burst and fan-out structure.** Agents fire parallel tool-calls: N simultaneous connections from one source with synchronized starts; humans open connections gradually. Features: new-flow

arrival rate per source (count-min sketch), concurrent flows per source, burst length in packets, destination diversity per time window. This family directly predicts the congestion harm being solved.

3. Packet-size and flow-shape statistics (survive encryption). Sizes of the first N packets of a flow (highly discriminative in encrypted-traffic ML), packet-length distribution moments/histogram bins, upstream/downstream byte ratio (agents: large structured JSON-RPC requests up, SSE streams down; human browsing: heavily download-asymmetric), flow duration. MCP-over-SSE signature: a long-lived flow carrying near-periodic small event/keepalive packets.

4. TLS handshake metadata (JA4 family — useful, never the anchor). The ClientHello is plaintext even in TLS 1.3: cipher-suite and extension counts/order, ALPN, GREASE, post-quantum key-exchange presence, SNI. Python/Go/Node agent stacks differ visibly from browsers — but agents driving real Chromium present browser-identical fingerprints (network-layer-only attribution: ~45–60%), and Encrypted ClientHello will erode SNI visibility over time. Treated as one input to the tree, with these limitations stated in the paper.

5. Explicit/cooperative signals (deployment-dependent). Web Bot Auth signature headers (RFC 9421) and MCP/A2A JSON-RPC protocol signatures travel inside TLS payloads — invisible mid-path, visible behind a TLS-terminating load balancer or on plaintext east-west datacenter traffic. Framing: the classifier runs on families 1–4 everywhere and gains family 5 as bonus features in TLS-terminated deployments.

Where each computation lives:

Feature	Computed in	Mechanism
IAT mean/variance, timing entropy	P4 fast path	Per-flow registers; EWMA on timestamp deltas
First-N packet sizes, length histogram	P4 fast path	Register array indexed by packet count
Up/down byte ratio, flow duration	P4 fast path	Directional byte counters
New-flow rate, fan-out per source	P4 fast path	Count-min sketch + window counters
SSE periodicity signature	P4 fast path	IAT regularity check on long-lived flows
JA4 fields (counts, ALPN, SNI)	P4 parser (1st pkt)	ClientHello parsing; full JA4 hash punted
Full JA4, Web Bot Auth verify, reclassification	SDN slow path	Controller inspects punts; updates tables

Three output classes, not two: human/interactive · interactive agent (latency-sensitive tool-calls) · bulk agent (crawling/batch). Time-critical agent traffic can receive priority treatment too — a stronger QoS story that also preempts the “anti-bot discrimination” objection.

8 · Implementation in Detail

8.1 · Data plane (P4 on BMv2, in Mininet)

The P4 program defines the full packet path. **Parser:** Ethernet → IPv4 → TCP/UDP; on a flow's first packet, parse TLS ClientHello fields (cipher/extension counts, ALPN, SNI length bucket). **Feature stage:** hash the 5-tuple to a flow index; update per-flow register arrays — last-seen timestamp, EWMA IAT mean and variance, packet count, directional byte counters, first-N packet-size slots; update per-source count-min sketch for new-flow rate and fan-out. **Classification stage:** the offline-trained decision tree is compiled into match-action tables using range matches over the register values (the Mousika/IIsy distillation recipe), emitting a 2-bit class label in packet metadata. **Action stage:** the label drives DSCP rewrite, egress-queue selection (three class queues with weighted round-robin), a per-class ECN marking threshold, and a per-class meter for admission control of new flows. **Human-protection**

mode: when the egress queue depth crosses a congestion threshold, scheduler weights shift to a protective preset that caps the aggregate agent share.

8.2 · Control plane (SDN slow path)

A Ryu (or ONOS) controller performs what cannot or should not run per-packet: it installs and updates the class→queue/weight policy; receives first-packet punts to compute the full JA4 string and, where visible, verify Web Bot Auth signatures; and runs the **entropy-shift reclassification loop** — periodically reading flow statistics and reassigning the class of flows whose timing behavior has drifted (an agent mimicking human pacing, or a misclassified human flow). The per-packet decision always stays in the device; the controller only refines it.

8.3 · ML workflow

Python + scikit-learn: train a shallow decision tree (depth ≤ 8) on the labeled traffic corpus (§9.1); export split thresholds; compile mechanically into P4 table entries. Feature importances are reported in the paper — timing and burst features are expected to dominate, reinforcing the design argument that line-rate-visible behavior suffices for class-level (not framework-level) discrimination.

8.4 · Traffic harness

Ground truth comes from real systems, not synthetic labels: an instrumented target (small web app + a real MCP server) receives traffic from actual agent frameworks (e.g., Browser Use, Claude with MCP tools, AutoGen, a Playwright-driven agentic browser) run in containers, with pcaps captured at the target — mirroring the methodology of the fingerprinting literature. AgentReady's WebMCP tool-call traffic serves as an additional realistic MCP workload source. Human traffic: replayed public anonymized traces (MAWI backbone captures; CAIDA passive traces) plus captured interactive browsing sessions for labeled human flows. Background load: iperf3 bulk flows.

9 · Evaluation Plan

9.1 · Setup

Mininet topology with the BMv2 switch at a choke point; link capacities scaled down (10–50 Mbps) so relative congestion behavior is preserved despite BMv2's software forwarding limits — standard, defensible practice in P4 evaluations, stated explicitly in the paper. Traffic mixes sweep agent share from 10% to 90% and burst intensity across a grid; each configuration runs with multiple seeds and reports confidence intervals.

9.2 · Baselines (four)

- FIFO / best-effort (no differentiation);
- Standard DiffServ with port/application-based marking;
- Per-flow fair queuing;
- An application-layer rate limiter (edge/WAF-style) — demonstrating that the in-network approach reacts faster and covers non-HTTP paths.

9.3 · Metrics and targets

- Human/interactive flow p50/p95/p99 latency under agent burst — **target: $\geq 30\%$ p99 reduction vs best baseline**;
- Agent tool-call completion time (interactive-agent class must not be starved);
- Classification precision/recall per class at line rate;
- Data-plane overhead: added per-packet latency (**target $< 5\%$**), table entries, register memory footprint;

- State scaling: accuracy and memory vs concurrent flow count (1k → 100k, with sketch-based approximations beyond exact-state capacity);
- Adversarial evasion: agents deliberately mimicking human pacing — report both the accuracy drop and the agent's throughput self-penalty, converting the “evasion is self-defeating” argument from claim to measurement.

The paper's centerpiece figure: human p99 latency held flat during an agent storm with the system on, versus the spike under every baseline.

10 · Phase-Wise Execution Checklist (15 weeks)

Phase 0 — Lock-in (this week)

- ☐ Freeze the one-liner and the four novelty axes; test every later design decision against them
- ☐ Freeze the contribution list (C1–C4) that the paper will claim

Phase 1 — Survey + related work (weeks 1–2)

- ☐ Build the related-work table: arXiv 2606.20910, 2606.30119, 2602.09606; Mousika/IIsy; Cloudflare Web Bot Auth / RFC 9421; DataDome/Akamai product docs; classic DiffServ/QoS literature
- ☐ Verify every remaining citation (incl. the operator/standards-bodies discussion line) against primary sources
- ☐ Write the introduction, motivation, and related-work sections of the paper now, while fresh

Phase 2 — Design (weeks 3–5)

- ☐ Freeze the line-rate feature set (nothing requiring payload reassembly)
- ☐ Design the P4 pipeline: parser → feature registers → tree-as-tables → action stage
- ☐ Design the SDN slow path: policy distribution + entropy-shift reclassification
- ☐ Draw the architecture diagram; finalize the three-class QoS policy and human-protection preset

Phase 3 — Prototype (weeks 6–10)

- ☐ Environment: Mininet + BMv2 + p4c toolchain; GitHub repo scaffolded from day one
- ☐ Traffic harness: real agent frameworks in containers against an instrumented MCP target (reuse AgentReady traffic); MAWI/CAIDA replay + captured browsing for human flows
- ☐ Train the decision tree offline; compile thresholds into P4 table entries
- ☐ Build incrementally: classification-only first (verify precision/recall), then wire the QoS action stage
- ☐ Get the centerpiece demo working: agent burst hits the link, human p99 stays flat
- ☐ Live per-class latency/throughput dashboard

Phase 4 — Evaluation (weeks 11–13)

- ☐ Run all four baselines across the agent-share × burst-intensity grid
- ☐ Collect latency percentiles, completion times, precision/recall, overhead, state-scaling and evasion results
- ☐ Multiple seeds + confidence intervals on every reported number
- ☐ Hit the anchor targets: ≥30% p99 reduction, <5% per-packet overhead

Phase 5 — Paper + demo (weeks 14–15)

- ☐ Finish the IEEE paper: methodology, four-baseline results tables, limitations (BMv2 emulation, DSCP domain scope, ECH erosion of SNI), future work (silicon-rate validation)

- Select target venue and format the submission accordingly
- Presentation + live demo of the human-traffic-protection scenario
- Repo cleanup: README, reproduction instructions, topology scripts, dataset/corpus documentation

Cross-cutting habits: keep a dated lab notebook of design decisions (feeds the methodology section and viva answers); re-check for newly published related work mid-semester — this space moved fast through 2025–2026, and week 8 is a better time to find a competing paper than week 15.

11 · Tech Stack

Layer	Technology	Role
Data plane	P4 ₁₆ on BMv2 (v1model)	Line-rate classification + QoS pipeline
Emulation	Mininet	Topology, links, hosts, choke point
Control plane	Ryu / ONOS (Python)	Policy, punts, reclassification loop
ML	Python, scikit-learn	Decision-tree training → table compilation
Agent traffic	Real agent frameworks in Docker + real MCP server; AgentReady WebMCP workloads	Ground-truth agent corpus
Human traffic	MAWI / CAIDA trace replay (tcpreplay); captured browsing; iperf3	Ground-truth human + background load
Measurement	Wireshark/tshark, Python (pandas/matplotlib)	Pcap analysis, latency percentiles, figures
Writing	IEEE LaTeX template; GitHub	Paper + reproducible artifact

12 · Key References (verified)

- Thales 2026 Bad Bot Report: Bad Bots in the Agentic Age (13th ed., Apr 2026) — traffic-share and attack-growth statistics.
- arXiv 2606.20910 — “Whose Agent Are You?”: multi-layer fingerprinting and attribution of autonomous web agents (97% with browser-level signals).
- arXiv 2606.30119 — “On the Internet, Nobody Knows You’re an LLM Bot”: network-layer-only agent attribution is weak (~45–60%); motivates the open problem.
- arXiv 2602.09606 — “When Handshakes Tell the Truth”: binary bot-vs-human detection from JA4/TLS fingerprints (~98.6%, AUC 0.998).
- arXiv 2510.07176 — AgentPrint: LLM-agent identification from encrypted traffic timing/volume (F1 0.866); privacy-risk framing.
- arXiv 2605.14786 — “Known By Their Actions”: fingerprinting LLM browser agents via UI traces.
- arXiv 1909.05680 — pForest: in-network inference with random forests at line rate (Tofino-validated).
- arXiv 2205.08824 — Planter: automated mapping of ML models to programmable data planes (de-facto mapping scheme).
- Mousika (INFOCOM 2022) — knowledge-distilled binary decision trees for data-plane deployment (compilation recipe adopted here).
- IIsy / “Do Switches Dream of Machine Learning?” (HotNets 2019 line) — foundational ML→match-action mapping.
- NetBeacon; Akem et al., Int. J. Network Management 2025 — in-switch real-time (encrypted) traffic classification with tree models.

- ACC-Turbo (SIGCOMM 2022) — in-network aggregate clustering coupled to programmable scheduling for pulse-wave DDoS mitigation.
- NetMCP + SONAR (Li et al., 2025) — network-aware MCP tool invocation weighing semantics and real-time QoS.
- arXiv 2607.16066 — survey: LLM-powered agentic AI for 5G/6G architectures, protocols, standardization (MCP/A2A in management planes).
- RFC 9421 / Web Bot Auth (Cloudflare, IETF) — cooperative cryptographic agent identification signals.